5 - Search

The term **set** with no adjectives does not generally imply order. In essence, a set or specifically an **unordered set** is just a pool of data with no structure. If a set has some structure and therefore some order imposed on it, it is typically referred to as an **ordered set** or by other terms that suggest the nature of the ordering imposed. For example, an **array** suggests a set that has positional order. There are a variety of ways to order and structure a set which we will explore further in the coming weeks. Many of these specific structures will be given names that will describe the nature of their ordering, structuring, and the rules by which they allow interactions and by which they behave.

Search is the process of locating an element within a set ordered or otherwise. It is often a two part question:

1. **existential** - "Does the value exist in the input set?"
2. **accessible** - "If it exists, retrieve its value or retrieve a reference to it" or "give access to it"

The accessible question is moot if the existential question fails, so the existential question is both the most basic and most important question to focus on when performing search on a set of data. Our analysis of search will assume the worst case scenario which is a failure to succeed with the existential question. In other words, when we perform a time complexity analysis, our analysis will assume search does not find the element in the set given the strategy and ordering used.

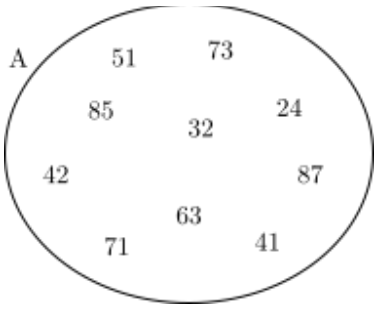
> **Worst Case Analysis**: Worst case does not mean that we intentionally design a poorly performing data structure. Worst case analysis is not analysis of a faulty design, it is an analysis of a data structure under its worst data conditions. This is a reflection on the fact that programmers control design but users generally control the data. Programmers have a variety of choices when designing data structures and it is the programmer's responsibility is to make good decisions on that design to ensure the best performance under expected data conditions. Worse case analysis should be performed on that design, not on an arbitrary, poor design that is incapable of meeting the necessary application requirements

Search is one of the most common computational operations that programmers take for granted. We should be concerned with the time complexity hidden behind a method like `.find(value)` if it is at the heart of one of our own processes and a search operation can come in many names: `search`, `find`, `contains`, `get`, or even list operators like braces, `[i]`, or parentheses, `(j)`. It is important to remember that the time complexity of any method we use scales the time complexity of our own algorithm and this becomes a multiplicative term to the time complexity of our algorithm. For implementations that must be efficient, we need to anticipate any implicit costs and minimize the use of any especially costly, redundant function calls.

Search Algorithms

Search requires a search set A, a search value e, and determines whether or not e is a member of A. Let `search(A, e)` be the general form for a search function where the first parameter, A, is the search set and the second parameter, e, is the search value.

For example, let set A, contain 73,51,41,85,32,63,24,42,87,71



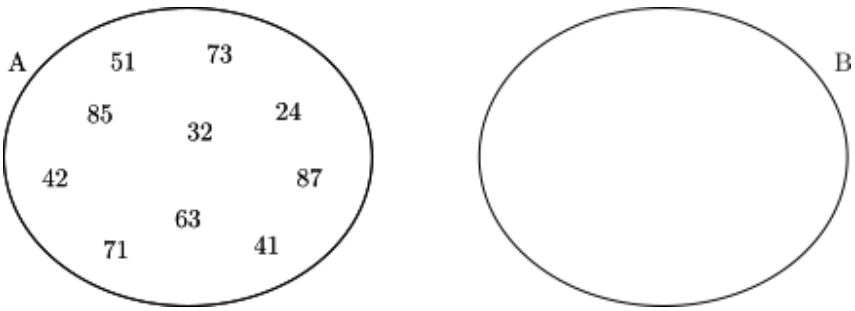
Let element $e = 41$ and let $e' = 93$, `search(A, e)` would at the least return `true` because 41 is a member of set A while `search(A, e')` would at the least return `false` because 93 is not a member of set A.

Random Sampling Search

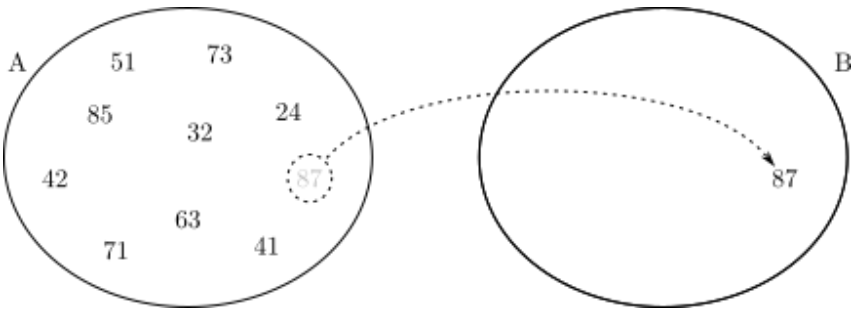
One search strategy is to randomly sample an element from the set and compare that element to the search value. Random sampling is like drawing tiles from an opaque bag where you may not peek inside the bag in any way. There are a number of issues with this brute force strategy particularly if we are required to return selected tiles to the bag as this will likely result in an infinite process.

How is an infinite process possible here? Simple, assume the element is not in the set. If there is no tracking of previously sampled elements as there is when we return the tile to the bag, there is no way to determine whether or not all elements have been sampled from the set. Without the ability to know when all the elements have been sampled, it is not possible to know when to end the process and therefore the efficiency is $O(\infty)$.

To ensure the process exits in a predictable amount of time, the process could instead divide the set into two subsets: the unsampled search set A, and the previously sampled set B. The unsampled search set represents tiles still in the bag and the sampled set represents tiles that we do not return to the bag once we have removed them.



Sampling for search is only performed on the unsampled set and when an element is sampled, it is moved from the unsampled set to the sampled set thus decreasing the unsampled set by one with each sampling.



This process guarantees that all elements in the set will be sampled and compared, that no element will be sampled and compared more than once, and that the process will exit in a predictable amount of time.

Mathematical Analysis

Recall that the worst case for search using this two set random sampling approach is that the tile does not exist in the original unsampled search set A, a set containing n elements. If an element can only be sampled once, it must be removed from set A and placed into set B when it is sampled; therefore, one new element is guaranteed to be sampled with each comparison. The maximum number of comparisons will be n because it will take n total samples (comparisons) to determine that the search element does not exist in the original search set A.

$$f(A_n) = \underbrace{1 + 1 + \cdots + 1}_{n}$$

This leads to a Big O analysis of $O(n)$ and a linear time complexity for random sampling in the worst case where already sampled elements are removed from the search set

Linear (Iterative) Search

Random search is inefficient and generally requires more work to implement than linear search in computing. Why? When using an array to represent a set, we are imposing a structural order on the set even if we are not imposing any additional order such as a lexicographic order. The structural order arises naturally because the array is ordered by position which we generally call the array index. It takes more work to treat the data as an abstract pool of data in a computer, so by default the iterative strategy is more natural and immediately accessible if data is stored in an array. Because linear search is exhaustive in the same way our two-set random sampling strategy was exhaustive, *i.e.* neither can determine if an element does not exist in the set unless every element in the set is sampled, linear search is also a brute force strategy. Because linear search is an iterated function or, more programmatically, it is implemented with a loop that iterates using the same operations repeatedly over all of the input data, it is also called iterative search.

Given an array of data A, containing n elements, with no additional order other than the natural positional order imposed by the array structure:

51	32	73	63	42	85	71	41	87	24
0	1	2	3	4	5	6	7	8	9

Iterative search operates by scanning the array from left-to-right. At each index, the algorithm compares the array element at that index a_i with the search element e. If the array element matches the search element, $a_i = e$, then the search element e exists in the set A, and the search was successful. If the process reaches the end of the array without ever finding the condition $a_i = e$, the search is a failure and e must not exist in the array A.

Mathematical Analysis

The worst case for search is that the element being searched for does not exist in the set. For a brute-force, iterative search, to confirm that a value does not exist in the set, it requires comparing the element in question to every value in the set. The mathematics is pretty simple:

Given an array A as input, n is defined as the number of elements in the array or simply as the size of A. As in random sampling with sample removal, linear search samples a single element each step and in the worst case it must sample all elements to confirm an element does not exist.

$$f(A_n) = \underbrace{1 + 1 + \cdots + 1}_{n}$$

Therefore, the algorithmic time complexity of linear search is $O(n)$.

As mentioned above, we generally base our analysis of the worst case, but let's consider other cases to better understand why worst case analysis is used.

Best Case

For the best case, the time complexity would be constant time, $O(1)$, because the best case is predicated on the assumption that we would always find our element in one comparison; however, what is the likelihood of the best case?

The apparent probability that the best case arises in linear search is $\frac{1}{n}$ which gets less and less likely as the size of n increases. Consider that if n is sufficiently large, the likelihood of the best case arising approaches 0, so the best case is highly improbable if not impossible for large data sets.

Further, consider that the best case approximation above assumes that the search value exists in the search set; however, this assumption cannot always hold and may be exceedingly rare in real data. Given a near infinite number of candidate elements and a fixed sized search set, there are far more elements NOT in the search set than those that exist in the search set which means that the probability of any given value being in the search set again approaches zero unless the domain of the universal set is relatively small. This means that the best case is extremely biased and virtually impossible so it is a poor representation of the efficiency of the algorithm under normal conditions.

Average Case

For the average case, the time complexity would be some average of all searches within the search set. Analyzing this might require we make some assumption like assuming that the search value must exist in the search set. If this assumption holds, then sometimes the search value is in the first place we look, sometimes it is in the second place we look, sometimes it is in the last place we look. If we assume a uniform probability distribution of these searches, then the likelihood of the search value being in any given position is $\frac{1}{n}$ and the number of operations to locate an element is equal to its position in the array. In other words, the average number of operations is:

$$\frac{1 + 2 + \ldots + (n-2) + (n-1) + n}{n}$$

We simplify the equation to:

$$\frac{\cancel{1} + \cancel{2} + \ldots + (n \cancel{-2}) + (n \cancel{-1}) + n}{n}$$

Which results in the elimination of half the terms leaving the equation:

$$\frac{\overbrace{n + \cdots + n}^{\frac{n}{2}}}{n}$$

Repeated addition of the same term is the definition of multiplication, so the above further simplifies to:

$$\frac{n}{2} \cdot \frac{n}{n}$$

Which then reduces to:

$$\frac{n}{2}$$

In other words, the time complexity is $O(n)$, so we do get a definitive average case time complexity analysis; however, it turns out to be the same as our worst case analysis and computing it requires both additional work and a more detailed mathematical analysis. On top of this, our average case above does not even take into consideration the more realistic data scenario that the value we are searching for does not exist in the search set as argued above. So the average case is at best $\frac{n}{2}$ and at worst n but is most likely worse than $\frac{n}{2}$ and likely trends towards n given realistic data. It is also difficult to precisely know what it should be, it is hard to compute, and it is trending towards the worst case anyway.

Pseudocode

Iterative Search (Existential):

```
search(A, e) → search set A, search value e : boolean, true if e found in A else false
    for v in A :
        if v == e :
            return true

    return false
```

Iterative Search (Accessible):

```
search(A, e) → search set A, search value e : index of the value in A if found else -1
    for i = 1 to n | n is size of A :
        if A[i] == e :
            return i

    return -1
```

Binary Search

Binary search is an alternate search strategy that uses a **divide and conquer** approach. Divide and conquer relies on structured data and allows us to discard data that fails to meet general criteria which means we do not need to exhaustively search every element in order to know whether or not the search element exists in the search set.

Consider this example:

We need to find S in the English alphabet. Because of the properties of the alphabet, we know that M is the middle letter of the alphabet. From these same properties, we know that S follows M. We can divide the alphabet in half centered around M, *i.e.* into the sets A-L, M, and N-Z, and compare S to M. Since S is not M and we know that S follows M, S cannot exist in the set A-L, and we can discard A-L entirely in any subsequent search involving S. We repeat this process on the remaining set N-Z, and so forth. If we reach the point where we are left searching only one subset of only one letter, the existential question becomes trivial, *i.e.* is the search value is either equal to that letter or it is not, and this can be determined in one step.

The key to binary search is that the data needs to be in a **lexicographic order** or in other words if the data is an array it needs to be structured by some known order such that it is "in order". It is the relationship between the position of an element in the ordered set and the relationship of elements to one another based on the lexicographic order that allows fast searching using binary search.

In the example above, the lexicographic order is the alphabetical order and from this we know that A is before B, C, etc., that Z is after Y, X, etc., and that M is after L but before N, etc. We also know that the middle position is 26/2 or the 13th letter, M. If we are searching for a search value that precedes M, there is no need to search in the region following M if the data is in alphabetical order which means we can completely ignore any elements following M if the search element precedes M.

Mathematical Analysis

Again, the worst case for search is that the element being searched for does not exist in the set. Because binary search samples only a few elements in its process, it is more efficient than iterative search; however, it is also less intuitive as to how to arrive at our estimate.

At each step, we divide the remaining search set in half. In other words for the initial search set of size n, we divide the set into two equal sized sets of size $\frac{n}{2}$ and discard one of these subsets. This requires only a single operation because we take advantage of the efficiency offered by sampling any index in an array in constant time. For the remaining subset of size $\frac{n}{2}$, we divide it into two equal sized sets of size $\frac{n}{4}$ and again discard one of these subsets using one operation. We continue to repeat this subdividing and discard process until we reach a subset of size 1. The progressive size of subsets that results is:

$$n, \frac{n}{2}, \frac{n}{4}, \frac{n}{8}, \cdots, 4, 2, 1$$

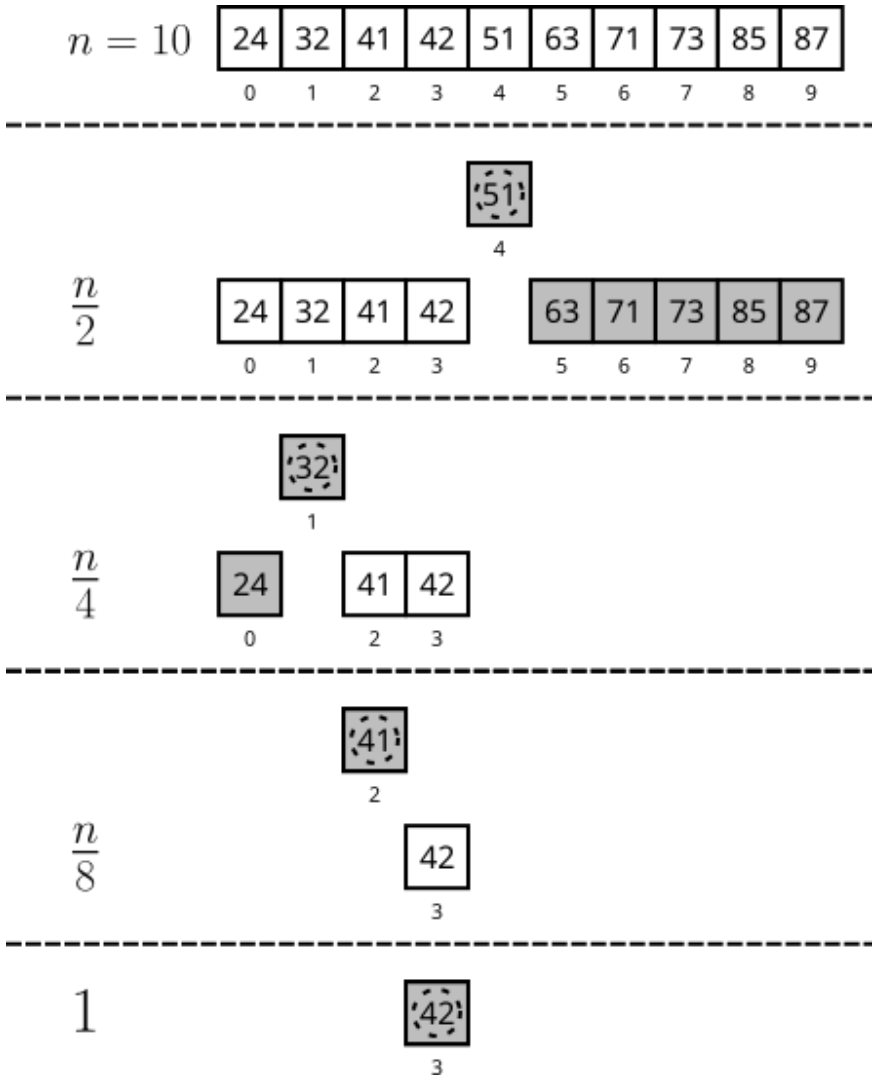
The number of operations is simply a question of how many terms are in the above description. So we need to answer what function predicts the decay of n to 1 if the decay rate is $\frac{1}{2}$ of the size of the remaining subset at each step. The answer is:

$$\underbrace{n, \frac{n}{2}, \frac{n}{4}, \frac{n}{8}, \cdots, 4, 2, 1}_{\log_2 n}$$

In other words, the number of terms defines the efficiency and the efficiency of binary search in the worst case is:

$$O(\log_2 n)$$

Consider an ordered search set with 10 elements containing the following values: 24, 32, 41, 42, 51, 63, 71, 73, 85, 87. If we search for a search value of 45 in the search set using binary search, the process is illustrated below.



The grayed out boxes represent values that are eliminated from further consideration in the search set at each step. The dashed circle represents the middle element of the remaining search set at each step and the single value compared to the search value at each step. As a result, this process requires a total of four comparisons to confirm that the search value 45 does not exist in the search set. At this point we would need to determine the relationship between the number of elements in the search set, n and the number of circles. This relationship was predicted and modeled by our mathematical analysis above, but the general relationship, considering that this is a discrete problem and $2^3 < n < 2^4$ where 4 is the number of comparisons, then the efficiency is logarithmic or $O(\log_2 n)$.

Here is another way to look at it. Binary search repeatedly reduces the search space with initial size n by half until the search problem is simply comparing whether or not one remaining element is the search value.

If k represents the number of times the search space is reduced in size by $\frac{1}{2}$:

$$\underbrace{\frac{1}{2} \times \frac{1}{2} \times \cdots \times \frac{1}{2}}_{k}$$

Simplified as:

$$\frac{n}{2^k}$$

We are trying to solve the above equation for when there is one remaining element in the search set because any remaining operations are constant time for a set of size 1. We can now formulate the problem:

$$\frac{n}{2^k} = 1$$

Recall that we are asking "how many reductions" which is what k represents, so our goal is to solve for k to find out how many times this process needs to be repeated until it reaches the end:

$$n = 2^k$$

$$2^k = n$$

Cancel the base to isolate k and answer our question:

$$\log_2 2^k = \log_2 n$$

$$k = \log_2 n$$

So, in the worst case where the search value does not exist in the initial ordered search set, it takes at most $\log_2 n$ reductions until the search set is reduced to a set of one element leaving us only one question as to whether or not that one element is the search value. Therefore the time complexity of binary search where n is number of elements in the array is:

$$O(\log_2 n)$$

Because this is the maximum number of comparisons necessary to determine whether or not an element exists within an ordered array when we use this binary search algorithm.

Pseudocode

Binary Search (Existential):

```
search(A, v, l, r) → ordered set A, search value v, search region [l,r]
                   : true if v is found in A, else false
    if l > r :
        return false

    mid = (l+r) / 2

    if A[mid] == v :
        return true
    if A[mid] > v :
        return search(A, v, l, mid-1)
    if A[mid] < v :
        return search(A, v, mid+1, r)
```

Binary Search (Accessible):

```
search(A, v, l, r) → ordered set A, search value v, search region [l,r]
                   : non-negative index of v if v in A else -1
    if l > r :
        return -1

    mid = (l+r) / 2

    if A[mid] == v :
        return mid
    if A[mid] > v :
        return search(A, v, l, mid-1)
    if A[mid] < v :
        return search(A, v, mid+1, r)
```

Generic Functions

The above algorithms are generically defined in pseudocode which is intentionally used to avoid the constraints of type imposed by Java. This raises the question of whether we can similarly define algorithms and data structures in Java such that type is not strictly defined in the implementation. In other words, can we define a search algorithm such that type is another parameter instead of a fixed requirement and the answer is yes by using generics. There are both generic functions and generic classes but we will focus on generic functions here.

A generic function accepts a type as a parameter. Typically we use the `T` symbol to indicate a generic type (we can use other symbols, but by convention we typically start with `T`). Syntactically, the angled braces along with an inscribed `T`, *i.e.* `<T>`, before the return type signals to the compiler that the function is a generic function. For example, the following function `genericFun` is an example of a generic function:

```
    public static <T> void genericFun( T x ) {
        System.out.println( x );
    }
```

The `<T>` preceding the `void` signals that `genericFun` is a generic function. The `T` in the parameter list preceding the variable `v` holds the place of the type of `x`. The generic type must be a reference type in Java passed to `genericFun` at compile time. It is important to note that a generic type can only work on reference types in Java, so primitive types cannot be directly used with generics. This might seem like a serious oversight; however, Java provides "wrapper" classes for each primitive type, *e.g.* `class Integer` for `int`, `class Float` for `float`, `class Character` for `char`, `class Boolean` for `boolean`, etc. For example, using `genericFun` just requires calling the method with a value represented with a reference type:

```
class GenericFunEx {

    public static void main( String[] args ) {
        genericFun( Integer.valueOf(5) );
        genericFun( Boolean.valueOf(false) );
        genericFun( new String("Hello") );
    }

    public static <T> void genericFun( T x ) {
        System.out.println( x );
    }
}
```

The above program outputs the following:

```
5
false
Hello
```

A generic type is known at compile time, enabling the compiler to perform syntax and type checking throughout compilation to enforce type safety.

Now reconsider the Iterative Existential search again, this time in Java using a generic function:

```
class GenericSearchIterativeEx {

    public static void main( String[] args ) {

        // generate an array of integers
        Integer[] arrInt = new Integer[5];
        for( int i = 0; i < arrInt.length; i++ ) {
            arrInt[i] = Integer.valueOf(i);
        }

        // use the generic searchIt function to search
```

```
        // for specific integers in the integer array
        System.out.println( searchIt( arrInt, Integer.valueOf(4)) );
        System.out.println( searchIt( arrInt, Integer.valueOf(6)) );

        // generate an array of floats
        Float[] arrFloat = new Float[5];
        for( int i = 0; i < arrFloat.length; i++ ) {
            arrFloat[i] = Float.valueOf(i);
        }

        // use the same generic searchIt function to search
        // for specific floats in the float array
        System.out.println( searchIt( arrFloat, Float.valueOf(4)) );
        System.out.println( searchIt( arrFloat, Float.valueOf(6)) );
    }

    public static <T extends Comparable<T>> boolean searchIt( T [] A, T v) {
        // iterate over all elements from beginning to end
        for( int i = 0; i < A.length; i++ ) {
            // use the comparison method implemented by any Comparable type
            // to compare whether the search value v is equal to the current element
            if( v.compareTo( A[i] ) == 0 ) {
                // if they are equal, the search value is found
                return true;
            }
        }
        // if the process gets here, the search value was never found in the
        // entire array
        return false;
    }
}
```

There may be some details that are not immediately obvious without further understanding of inheritance or other high level concepts and this is also not the only way to solve this problem; however, the above does address the question and allows us to write and maintain one function that implements this algorithm. We could do the same for binary search or an other algorithm using generic functions to allow any type of object to be supported with a single implementation. Similar principles allow us to implement generic data structures that ensure we can write one class to support any type of object.

Review

1. Why are we so concerned with the efficiency of search? Why can the efficiency of search act as a multiplier to the efficiency of virtually all other algorithms we develop or use? Give an example.
2. What is the existential question in terms of set membership and why does it define worst case analysis of search?
3. Why is worst case analysis used? Why not use best or average cases to analyze efficiency instead?
4. Summarize the strategy for Iterative Search

5. Summarize the strategy for Binary Search
6. What are the requirements for Iterative Search
7. What are the requirements for Binary Search
8. Define Pseudo-code for Iterative Search
9. Define Pseudo-code for Binary Search
10. Analyze the efficiency for Iterative Search
11. Analyze the efficiency for Binary Search